The `PageModel` class is generated automatically when you add a new Razor page to your application. It is named after the page, with the word *Model*, so the `PageModel` class for your Welcome page is `WelcomeModel`. `PageModel` classes are derived from the `PageModel` type, which has a lot of properties and methods related to working with HTTP requests within the `PageModel` class itself. Public `PageModel` properties are exposed to the Razor page through the inclusion of an `@model` directive that references the `PageModel` type.

3.4.1 Passing data to pages

You have many options for passing data to pages. The recommended approach is working with data in a strongly typed manner as a view model. There is another option which, while it is weakly typed, is also of use sometimes. You have already used this approach to pass the Welcome page's title to the layout in the previous chapter. It is called `ViewData`.

`ViewData` is a dictionary-based feature. Items are stored within the `ViewDataDictionary` as key-value pairs and accessed by referencing the case-insensitive string key of the `ViewData` property within the page. The following is how you used `ViewData` to assign the Welcome page's title:

```
ViewData["Title"] = "Welcome!";
```

This value is accessed in the layout page as follows:

```
<title>@ViewData["Title"] - WebApplication1</title>
```

The values within the `ViewDataDictionary` are `object` types, which means you can store anything you like there. Non-string types need to be cast to their correct types if you want to work with them (e.g., to call type-specific methods). This may not be necessary if all you want to do is render the values and